

INDIA.RUNS HACKATHON · DATA & AI CHALLENGE

Redrob Candidate Ranking System

Ranking 100,000 candidates the way a great recruiter would —
not by matching keywords, but by understanding who actually fits.

Submitted by: Aahana
Track 1 — Data & AI Challenge

System Summary

A 10-second overview for judges reviewing multiple submissions.

What	Result
Candidates processed	100,000
Ranking runtime (Stage 2)	Under 2 minutes — CPU only
LLM API calls at ranking time	Zero
GPU required	No
Honeypot profiles detected & excluded	60 out of 100,000
Ranking explainability	Evidence quotes from career history
Output	Top 100 recruiter-ready ranked shortlist
Format validation	PASSED

The Problem

When a recruiter posts a job, hundreds of resumes pour in. The standard response is to apply keyword filters — if the JD says 'FAISS', surface the candidates who wrote 'FAISS'. This feels logical. It is not.

The problem has two faces. First, strong candidates get missed — someone who spent three years building hybrid retrieval systems may never have written 'vector database' as a standalone phrase. The keyword filter never sees them. Second, weak candidates surface — someone who listed every AI buzzword in their skills section despite never having shipped anything real.

The Redrob JD said it directly: “Not by matching keywords, but by actually understanding who fits.” This was the entire design brief.

The dataset amplifies this deliberately. It contains keyword stuffers, honeypot profiles with impossible career histories, pure-services backgrounds dressed in AI language, and inactive candidates who look perfect on paper but haven't logged in for six months. A semantic-ranking baseline alone will fail on all of these. This submission is built around solving that.

The Approach

The system is built on one core insight: **rank based on what candidates have actually done, not what they have listed**. Every candidate has two representations — a skills section (keywords) and a career history (real work). This system uses the career history.

System Architecture

STAGE 1 — Offline Precompute	STAGE 2 — Fast Ranking (CPU, <2 min)
candidates.jsonl (100K records) ↓ Career narrative builder (job descriptions, not skills list) ↓ all-MiniLM-L6-v2 (local model, no API call) ↓ Saved embeddings to disk	Job description → embedded ↓ Cosine similarity vs all embeddings ↓ Rule-based scoring (hard req + behavioral + penalties) ↓ Score combiner (weighted formula) ↓ Top 100 with evidence quotes
Stage 1 runtime	~15 minutes (one-time precompute, offline)
Stage 2 runtime	Under 2 minutes on CPU — within 5-minute constraint
Hardware requirement	CPU only, 16GB RAM — no GPU, no external API

Scoring Design

Formula: final score = $0.40 \times \text{semantic} + 0.25 \times \text{hard_req} + 0.20 \times \text{behavioral} + 0.10 \times \text{location} - \text{penalties}$

Weight Justification

Component	Weight	Why this weight
Semantic fit	40%	The JD explicitly prioritises understanding experience over keyword matching — largest weight reflects this mandate directly.
Hard requirement evidence	25%	JD lists these as MUST-HAVE. Lower than semantic because term presence alone does not prove depth of experience.
Behavioral availability	20%	JD explicitly warned: a perfect-on-paper candidate inactive for 6 months is not actually hireable. Availability matters nearly as much as fit.
Location fit	10%	Preferred but not a hard requirement per JD — other India metros acceptable. Minor bonus, not a filter.
Penalties	-ve	Subtracted additively, not multiplicatively — one red flag reduces score proportionally without eliminating an otherwise strong candidate.

Trap & Honeypot Detection

The dataset documentation warned explicitly about honeypot profiles. The JD warned about keyword stuffers. Both were taken seriously and handled with distinct logic.

Real Honeypot Example (detected and excluded):

Field	Value	Why suspicious
Skill: FAISS	Proficiency: Expert	Expert with 0 months duration — impossible
Claimed experience	9.0 years	Career history totals only 24 months — 7+ year gap
Job descriptions	All buzzwords present	Generic text, no specific systems or scale mentioned
→ Action	Excluded entirely	Never appears in ranked output

Candidates are not penalised for working at consulting firms. The penalty applies only when the entire career history shows no evidence of product ownership, retrieval systems, ranking systems, or deployment experience despite extensive experience.

Why Beyond Cosine Similarity

A common semantic-ranking baseline computes cosine similarity between the JD and candidate embeddings, then returns the top results. This misses a critical signal: a candidate who looks strong on paper may not actually be available or hireable. The following comparison shows why multi-signal scoring matters.

Signal	Candidate A	Candidate B
Semantic fit (career narrative)	0.76	0.79
Hard requirement evidence	1.00	0.50
Behavioral availability	0.85	0.08
Location fit	1.00	1.00
Final Score	0.79	0.47
Rank	#1	Not shortlisted

Candidate B had a slightly higher semantic score — a pure cosine-similarity system would have ranked them higher. But Candidate B had not logged in for 7 months and had a 4% recruiter response rate. They are not actually available. Behavioral scoring surfaces this — cosine similarity cannot.

Key Differentiators

- 1. Career narrative, not skills list.** The embedding is built from actual job description text — what the candidate did, not what they listed. A skills section contributes a small fraction of the total text so it cannot dominate the embedding. This directly addresses the keyword-trap.
- 2. Evidence-grounded reasoning.** Every candidate in the output comes with a reasoning line quoting a real sentence from their career history. A recruiter reading rank 1 immediately sees the specific system this person built and where. This is verifiable — it builds trust.
- 3. Transparent, defensible scoring.** Each weight has an explicit rationale tied directly to JD language. The system is not a black box.
- 4. Synonym-aware hard requirement matching.** Checks across 25+ related terms — not just exact strings. 'Nearest neighbor index', 'vector store', 'BM25 + dense' all count the same as 'FAISS' and 'Pinecone'.
- 5. Deliberate trap-awareness.** Designed knowing the dataset has traps. Manual verification of the top 20 candidates was done after ranking. Synthetic template-like descriptions were caught and documented.

Ablation Study

Four configurations were compared to show each component adds measurable value. Relevance labels are proxy-based (3+ hard requirement term matches in career history) since no official ground truth was provided.

Configuration	NDCG@10	P@10	MAP@100
Config 1: Keyword matching only (baseline)	0.93	0.90	0.90
Config 2: Semantic similarity only	1.00	1.00	0.85
Config 3: Semantic + Hard requirement	1.00	1.00	1.00
Config 4: Full system (all signals)	1.00	1.00	0.99

Keyword-only baseline scores 0.93 NDCG@10 — high because the proxy label itself is keyword-based. The full system's advantage shows most clearly in MAP@100 (0.99 vs 0.90) — better ordering across the full shortlist, not just the top 10.

Limitations & Future Work

Limitations

Weights are manually designed, not learned. The scoring weights were selected through iterative reasoning against the JD mandate and validated via ablation, not trained on labeled data. A learning-to-rank approach could potentially optimise weights automatically.

Relevance labels for evaluation are proxy-based. No official ground truth was provided in the dataset. Evaluation metrics reflect system consistency against a proxy signal, not absolute ranking quality.

Career narrative quality depends on job description text. Candidates who wrote sparse or generic job descriptions may be under-represented by their embeddings, even if their actual experience is strong.

No fairness audit was conducted. The system does not explicitly use protected attributes (gender, religion, caste), but indirect proxies such as institution tier or location could introduce bias. A systematic fairness check would be the first step before production deployment.

Future Work

Learning-to-rank models	LightGBM Ranker or LambdaMART with recruiter feedback as training signal
Cross-encoder reranking	Run a cross-encoder on top-200 for more precise final ranking
Recruiter feedback loops	Use interview/hire outcomes to continuously improve ranking weights
Fairness monitoring	Systematic bias checks across demographics before production
Automatic weight learning	Replace manually set weights with a learned scoring function

Technical Stack & Files

Language	Python 3.x
Embedding model	all-MiniLM-L6-v2 via sentence-transformers (local, no API)
Vector operations	NumPy (cosine similarity, matrix operations)
Data handling	pandas, json, csv
Demo / sandbox	Gradio on HuggingFace Spaces (public URL)
Version control	GitHub (public repository)

precompute_embeddings.py	One-time offline precompute — builds career narratives and saves embeddings
rank_candidates.py	Main ranking script — scores all signals, writes submission.csv
ablation_study.py	4-configuration comparison with NDCG/P@10/MAP metrics
explore_data.py	Data exploration and trap pattern analysis
verify_top_candidates.py	Manual sanity check — full career details of top N candidates
API_CONTRACT.md	Input/output schema, edge case handling, constraints
submission.csv	Final ranked output — top 100 with score and reasoning

GitHub:

github.com/aahanachoudhary01/redrob-candidate-ranking

Demo: huggingface.co/spaces/aahanachoudhary01/redrob-candidate-ranking

INDIA.RUNS HACKATHON · DATA & AI CHALLENGE

Submitted by Aahana · Track 1 — Data & AI Challenge